Exercise 1: Ethiopian Coffee Ceremony

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage details of the Ethiopian coffee ceremony.

- Create a class to represent a coffee ceremony with attributes such as **ceremony name** and **number of cups** served.
- Apply ***Encapsulation*** so that the internal details of the class are hidden.
- Provide mechanisms to initialize, update, and display ceremony information.
- In a separate class, store multiple ceremonies in a collection and print their details.
- Demonstrate how one ceremony's cup count can be updated and retrieved through the encapsulated design.

Solution:

```
class CoffeeCeremony {
    private String name;
    private int cups;
    public CoffeeCeremony(String name, int cups) {
       this.name = name;
       this.cups = cups;
    }
    public String getName() {
       return name;
    }
    public int getCups() {
       return cups;
    }
    public void setCups(int cups) {
       if (cups > 0) {
          this.cups = cups;
       } else {
          System.out.println("Number of cups must be positive!");
       }
    }
    public void display() {
       System.out.println(name + " Ceremony - " + cups + " cups");
    }
}

import java.util.ArrayList;
import java.util.List;
public class CoffeeDemo {
    public static void main(String[] args) {
       List<CoffeeCeremony> ceremonies = new ArrayList<>();
       ceremonies.add(new CoffeeCeremony("Abol", 3));
       ceremonies.add(new CoffeeCeremony("Tona", 3));
       ceremonies.add(new CoffeeCeremony("Baraka", 3));
       for (CoffeeCeremony c : ceremonies) {
          c.display();
       }
```

```
        // Demonstrate encapsulation
        CoffeeCeremony abol = ceremonies.get(0);
        abol.setCups(4); // update cups
        System.out.println("Updated: " + abol.getName() + " - " +
                           abol.getCups() + " cups");
    }
}
```

Exercise 2: Ethiopian Traditional Food Management

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage Ethiopian traditional food details.

- Create a class to represent a traditional food with attributes such as **food name** and **preparation time (in minutes)**.
- Apply ***Encapsulation*** so that the internal details of the class are protected.
- Provide mechanisms to initialize, update, and display food information.
- In a separate class, store multiple foods in a collection and print their details.
- Demonstrate how one food's preparation time can be updated and retrieved through the encapsulated design.

Solution

```
class TraditionalFood {
    private String name;
    private int prepTime;

    public TraditionalFood(String name, int prepTime) {
        this.name = name;
        this.prepTime = prepTime;
    }
    public String getName() {
        return name;
    }
    public int getPrepTime() {
        return prepTime;
    }
    public void setPrepTime(int prepTime) {
        if (prepTime > 0) {
            this.prepTime = prepTime;
        } else {
            System.out.println("Preparation time must be positive!");
        }
    }
    public void display() {
        System.out.println(name + " - " + prepTime + " minutes");
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;

public class FoodDemo {
    public static void main(String[] args) {
       List<TraditionalFood> foods = new ArrayList<>();
       foods.add(new TraditionalFood("Injera", 60));
       foods.add(new TraditionalFood("Shiro", 30));
       foods.add(new TraditionalFood("Doro Wat", 90));

       for (TraditionalFood f : foods) {
          f.display();
       }
       // Demonstrate encapsulation
       TraditionalFood shiro = foods.get(1);
       shiro.setPrepTime(35); // update preparation time
       System.out.println("Updated: " + shiro.getName() + " - " +
       shiro.getPrepTime() + " minutes");
    }
}
```

Exercise 3: Fasting

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage fasting details in Ethiopia.

- Create a class that represents Ethiopian fasting traditions, including their **names** and **durations**.
- Apply ***Encapsulation*** so that the internal details of this class are protected and cannot be accessed directly.
- In a separate class, store multiple fasting traditions in a collection and print them out.
- Finally, demonstrate how one of the fasting details can be updated and retrieved through the encapsulated design.

Solution

```
public class Fast {
    private String name;
    private int days;

    public Fast(String name, int days) {
       this.name = name;
       this.days = days;
    }
    //Getter methods
    public String getName() {
       return name;
    }
    public int getDays() {
       return days;
    }
    // Setter methods
    public void setName(String name) {
       this.name = name;
    }
    public void setDays(int days) {
       this.days = days;
    }
    // Display method
    public void display() {
       System.out.println(name + " - " + days);
    }
}

import java.util.ArrayList;
import java.util.List;

public class FastingDemo {
    public static void main(String[] args) {
       List<Fast> fastingList = new ArrayList<>();
       fastingList.add(new Fast("Ramadan", 30));
       fastingList.add(new Fast("Abiy Tsom", 55));
       fastingList.add(new Fast("Dhul Hija", 10));
       fastingList.add(new Fast("Gena", 30));
```

```
        // Display all fasting info
        for (Fast f : fastingList) {
                    f.display();
        }
        // Demonstrate encapsulation: update values using setters
        Fast ramadan = fastingList.get(0);
        ramadan.setDays(29); // Suppose Ramadan is 29 days this year
        System.out.println("Updated: " + ramadan.getName() + " - " +
        ramadan.getDays());
    }
}
```

Exercise 4: Ethiopian Holidays and Festivals

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage Ethiopian holiday and festival details.

- Create a class to represent a holiday or festival with attributes such as **name** and **duration (in days)**.
- Apply *Encapsulation* so that the internal details of the class are hidden.
- Provide mechanisms to initialize, update, and display holiday information.
- In a separate class, store multiple holidays in a collection and print their details.
- Demonstrate how one holiday's duration can be updated and retrieved through the encapsulated design.

Solution

```
class Festival {
    private String name;
    private int duration;
    public Festival(String name, int duration) {
       this.name = name;
       this.duration = duration;
    }
    public String getName() {
       return name;
    }
    public int getDuration() {
       return duration;
    }
    public void setDuration(int duration) {
       if (duration > 0) {
          this.duration = duration;
       } else {
          System.out.println("Duration must be positive!");
       }
    }
    public void display() {
       System.out.println(name + " - " + duration + " days");
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;

public class FestivalDemo {
    public static void main(String[] args) {
       List<Festival> festivals = new ArrayList<>();
       festivals.add(new Festival("Timket", 3));
       festivals.add(new Festival("Meskel", 1));
       festivals.add(new Festival("Eid Al-Fatir", 1));
       festivals.add(new Festival("Eid Al-Adha", 1));

       for (Festival f : festivals) {
          f.display();
       }
       // Demonstrate encapsulation
       Festival eid = festivals.get(3);
       eid.setDuration(3); // update duration
       System.out.println("Updated: " + eid.getName() + " - " +
                           eid.getDuration() + " days");
    }
}
```

Exercise 5: Ethiopian Music Scales (*ቅኝት*)

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage Ethiopian *ቅኝት* (poetic/musical styles) details.

- Create a class to represent a *ቅኝት* style with attributes such as name and origin region.
- Apply *Encapsulation* so that the internal details of the class are hidden.
- Provide mechanisms to initialize, update, and display style information.
- In a separate class, store multiple *ቅኝት* styles in a collection and print their details.
- Demonstrate how one style's origin can be updated and retrieved through the encapsulated design.

Solution

```
class Kignit {
    private String name;
    private String origin;

    public Kignit(String name, String origin) {
        this.name = name;
        this.origin = origin;
    }
    public String getName() {
        return name;
    }
    public String getOrigin() {
        return origin;
    }
    public void setOrigin(String origin) {
        if (origin != null && !origin.isEmpty()) {
           this.origin = origin;
        } else {
           System.out.println("Invalid origin!");
        }
    }
    public void display() {
        System.out.println(name + " - Origin: " + origin);
    }
}

import java.util.ArrayList;
import java.util.List;

public class KignitDemo {
    public static void main(String[] args) {
        List<Kignit> styles = new ArrayList<>();
        styles.add(new Kignit("Ambasel", "Wollo"));
        styles.add(new Kignit("Zelesegna", "Shewa"));
        styles.add(new Kignit("Bati", "Wollo"));
        styles.add(new Kignit("Anchihoye", "Gondar"));
```

```
        for (Kignit k : styles) {
           k.display();
        }

        // Demonstrate encapsulation
        Kignit ambasel = styles.get(0);
        ambasel.setOrigin("North Wollo"); // update origin
        System.out.println("Updated: " + ambasel.getName() + " -
                            Origin: " + ambasel.getOrigin());
    }
}
```

Exercise 6: Ethiopian Musical Instruments

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage Ethiopian musical instrument details.

- Create a class to represent a musical instrument with attributes such as **name** and **category** (string, wind, percussion).
- Apply ***Encapsulation*** so that the internal details of the class are protected.
- Provide mechanisms to initialize, update, and display instrument information.
- In a separate class, store multiple instruments in a collection and print their details.
- Demonstrate how one instrument's category can be updated and retrieved through the encapsulated design.

Solution

```
class Instrument {
    private String name;
    private String category;

    public Instrument(String name, String category) {
       this.name = name;
       this.category = category;
    }
    public String getName() {
       return name;
    }
    public String getCategory() {
       return category;
    }
    public void setCategory(String category) {
       if (category != null && !category.isEmpty()) {
           this.category = category;
       } else {
           System.out.println("Invalid category!");
```

```
        }
    }
    public void display() {
        System.out.println(name + " - Category: " + category);
    }
}

import java.util.ArrayList;
import java.util.List;

public class InstrumentDemo {
    public static void main(String[] args) {
        List<Instrument> instruments = new ArrayList<>();
        instruments.add(new Instrument("Masenqo", "String"));
        instruments.add(new Instrument("Washint", "Wind"));
        instruments.add(new Instrument("Kebero", "Percussion"));
        instruments.add(new Instrument("Begena", "String"));
        for (Instrument i : instruments) {
            i.display();
        }
        // Demonstrate encapsulation
        Instrument begena = instruments.get(3);
        begena.setCategory("Sacred String"); // update category
        System.out.println("Updated: " + begena.getName() + " -
                                    Category: " + begena.getCategory());
    }
}
```

Exercise 6: Ethiopian Heritage

Instruction

Write a Java program to exercise *Encapsulation*. The program should manage Ethiopian heritage details.

- Create a class to represent an musical instrument with attributes such as **name** and **location**.
- Apply ***Encapsulation*** so that the internal details of the class are protected.
- Provide mechanisms to initialize, update, and display cultural and natural heritage information.
- In a separate class, store multiple heritages in a collection and print their details.
- Demonstrate how one heritage's location can be updated and retrieved through the encapsulated design.

Solution

```
class Heritage {
    private String name;
    private String location;

    Heritage(String name, String location) {
       this.name = name;
       this.location = location;
    }
    public String getName() {
       return name;
    }
    public String getLocation() {
       return location;
    }
    public void showInfo() {
       System.out.println(name + " is located in " + location + ".");
    }
}
import java.util.ArrayList;
import java.util.List;

public class HeritageDemo {
    public static void main(String[] args) {
       List<Heritage> heritages = new ArrayList<>();
       heritages.add(new Heritage("Simen Mountains", "Gondar"));
       heritages.add(new Heritage("AxumObelisks", "Axum"));
       heritages.add(new Heritage("Harar Jegol", "Harar"));
       for (Heritage i : heritages) {
          i.showInfo();
       }
       // Demonstrate encapsulation
       Heritage debark = heritages.get(0);
       debark.setLocation("Debark"); // update category
       System.out.println("Updated: " + debark.getName() + " -
                                  Location: " + debark.getLocation());
```

```
        }
}
```